

🔧 专业技能

技术栈

熟悉 springboot，MyBatis 等主流框架技术，熟悉 SpringIOC、AOP 和 SpringMVC 等；
在实习项目中接触了DDD分层架构，理解Domain层与应用层的职责划分；
掌握 MySQL 数据库，了解事务管理，SQL 优化，索引设计；熟悉常用数据结构与算法；
熟悉 Java 集合框架，多线程等相关内容，了解锁机制，Threadlocal，IO 模式以及常见设计模式；
了解Redis基础数据结构的使用场景；在项目中使用Redis做接口鉴权Token缓存，了解缓存过期策略和淘汰机制；
了解 JVM 内存结构、类加载机制、垃圾回收机制；熟悉常用的数据结构，了解算法设计基础；
在实习项目中使用RabbitMQ处理异步任务分发，了解消息确认机制（ACK）和基本的消息可靠性保障方式；
具有 LangChain4j ReAct Agent 开发经验；理解 MCP 协议规范，从零实现过 MCP Server；

📅 工作经历

杭州磐太科技有限公司

2026.3.9-至今

java后端实习生

项目：古树名木普查（GSDC）

- 参与巡检编号生成方案的技术选型与实施。多巡检员并发提交时编号需全局唯一且连续，团队评估后采用 PostgreSQL Advisory Lock 实现行级互斥生成业务编号，参与实施验证；
- 负责草稿/提交双模式校验策略的设计与实现。外业采集网络不稳定，设计了草稿仅校验必填项、提交执行完整业务规则的分层校验方案，降低弱网环境下数据丢失率；
- 参与 DDD 分层落地的实践过程，理解业务规则内聚至领域实体层的设计思路，掌握从业务需求到代码实现的完整流转；
- 负责省级编号字段端到端改造。后端沿 DDD 四层架构补齐字段写入链路（DTO → Command → 领域上下文 → MyBatis SQL），前端解除只读锁定并新增必填校验。涉及 17 个文件，48 项单元测试全部通过；
- 通过 EXPLAIN 分析查询计划定位全表扫描问题，为古树数据表创建 GIST 空间索引，查询计划从 Sequential Scan 优化为 Index Scan；优化 ZServer WMS 图层配置，去除冗余图层并设置缩放分级加载，显著降低了WMS请求数，解决地图缩放卡顿问题；

技术栈：Java，Spring Boot，DDD，PostgreSQL+PostGIS，MyBatis-Plus，RabbitMQ，OAuth2+JWT，OSS·MinIO·S3

📁 项目经历

Java MCP Server

项目背景：从零实现一个轻量 MCP 协议服务器，不依赖任何 MCP SDK，为 Claude Desktop 等客户端提供代码分析工具。

项目亮点：

- 按 MCP 协议规范手写 JSON-RPC 2.0 编解码器，处理 initialize、tools/list、tools/call 三类核心方法，基于 AtomicReference<State> 无锁状态机管理连接生命周期（INIT → READY → SHUTDOWN），通过 stdin/stdout JSON 流与客户端通信；
- 设计 @Tool / @ToolParam 注解驱动的工具注册框架，通过 Method.getAnnotation() + 反射 自动完成工具发现、JSON Schema 生成、参数按名解析与类型转换，新增工具只需编写一个注解方法，无需修改注册逻辑；
- 实现 7 个代码分析工具：代码统计（支持 22 种源文件扩展名，按语言分类统计行数/注释/空行）、依赖分析（识别 Maven/Gradle 依赖树）、复杂度报告（类/方法数量、最大文件行数）、工程化检查（CI/CD、Docker、测试、日志等 9 项指标检测）；
- 定义了协议层、工具层、传输层的异常分类，统一转换为 JSON-RPC 2.0 标准错误码返回客户端；
- 编写 30 个单元测试（JUnit 5），覆盖协议编解码、工具调用链路、异常处理与传输层收发；

技术栈：Java 17，Jackson，Java Reflection API，PDFBox，SLF4J，JUnit 5，Maven

DiffGuard

项目背景：个人开发的一款基于 LLM 的自动化代码审查工具，对 Git Diff 进行分析，识别潜在的安全漏洞和逻辑问题，支持 Git Hook（pre-commit/pre-push）本地拦截和 GitHub Webhook PR 自动审查。

项目亮点：

- 采用两轮审查架构，首轮生成 diff 摘要压缩上下文，次轮按文件分发并行审查，按路径+行号去重合并输出结构化 JSON 报告；
- 实现代码片段检索功能，离线模式基于关键词分词匹配（按空格/驼峰/下划线拆分），在线模式调用Embedding API计算向量相似度，通过配置开关切换两种模式；
- 使 LLM 能够自主获取跨文件审查所需的代码上下文，基于 LangChain4j ReAct 框架实现 Agent 工具调用，定义调用链查询、AST 节点获取、语义检索、文件白名单校验 4 类工具方法，由 LLM 在审查过程中动态选择调用；
- 实现指数退避重试（429/5xx）与断路器跳过（402），引入 Caffeine 缓存已审查文件的 diff hash，相同变更不重复调用。

技术栈：LangChain4j，JavaParser，JGit，Caffeine，Javalin，picocli，Jackson，JUnit 5，Mockito